SIH 2026

TECHNICAL REQUIREMENTS DOCUMENT

Food Traceability + Authenticity + Consumer Accountability + Risk Response Platform

TRD v2.0 — Detailed Base Model Engineering Baseline

Scope decision: build the complete core platform now; keep EPCIS, event streaming, IoT, advanced privacy, AI/ML, Advanced Risk & Recall, and Advanced Business Intelligence as later modules.

Engineering principle: *Blockchain is the trusted multi-organization layer. It is not the whole application.*

This document is based on the uploaded PRD, the earlier project blueprint/core-story documents, the supplied research material, and the technical decisions made during the project discussions. Source-supported facts are preserved; implementation choices that were not frozen by the source material are explicitly presented as engineering recommendations.

1. TRD Objective

The purpose of this TRD is to freeze the technical Base Model that the team should implement first. The uploaded PRD already defines the product requirements, workflows, actors, data entities, architecture responsibilities, acceptance criteria and phased scope. This TRD turns that product baseline into an engineering plan: which components exist, what each component owns, what is stored where, which operations become Fabric transactions, how PostgreSQL and Fabric stay consistent, how QR verification works, how feedback enters the system, and how the first Risk Propagator works. ■filecite■turn9file0■L1-L10■

The supplied research/blueprint also supports the separation of responsibilities: Hyperledger Fabric as a permissioned trust layer, chaincode for trusted rules, PostgreSQL for operational/query data, Redis for caching, IPFS for large evidence, and a backend Risk Propagator for bidirectional lineage analysis. ■filecite■turn10file5■L1-L1■

The team should not begin by implementing every future feature. The first milestone is one complete, testable product journey: source registration → validation → transfer → transformation → final unit → consumer traceability → incident → upstream/downstream impact analysis → basic action/recall scope.

2. Frozen Base Model Scope

The Base Model includes everything required for a complete working core except the explicitly deferred modules listed later. It therefore includes not only traceability, but also the first usable version of feedback, accountability, Risk Propagation, basic recall/blocking, basic stakeholder dashboards, evidence handling, outer QR and the inner-authenticity interface/mechanism required for the demo.

Base capability	Base Model decision
Organizations + users	Included — known participants, roles, authentication and RBAC
Product / Batch / Unit	Included — unit identity where required
Lifecycle + events	Included — controlled transitions and append-only business history
Product lineage	Included — one-to-many and many-to-one where process requires
Hyperledger Fabric	Included — permissioned trust layer
Chaincode	Included — critical shared business rules
PostgreSQL	Included — operational + lineage/query model
Redis	Included — cache only
IPFS	Included — evidence layer with persistence/pinning strategy
Outer QR	Included — traceability reference
Inner authenticity	Included — functional V1 mechanism/interface; final advanced cryptography remains separately hardenable
Consumer feedback	Included — unit/batch-linked incident reporting
Accountability + pattern escalation	Included — configurable, rule-based V1
Risk Propagator	Included — bidirectional core traversal
Basic recall/block workflow	Included
Basic stakeholder dashboard	Included

3. Explicitly Deferred Modules

The following are **not part of Base Model V1**: EPCIS compatibility, event streaming/Kafka, IoT sensor ingestion, advanced privacy/selective disclosure/private-channel architecture, AI/ML analytics, Advanced Risk & Recall, and Advanced Business Intelligence. They should be designed as plug-in modules around the stable identity, event and lineage foundation.

Later module	What it will add
EPCIS	Standardized interoperable supply-chain event semantics
Event streaming	Kafka/high-volume asynchronous event ingestion

Later module	What it will add
IoT	Temperature, humidity, GPS and other sensor evidence
Advanced Privacy	Private data collections, selective disclosure and stronger isolation
AI/ML	Anomaly detection, risk prediction, complaint clustering and decision support
Advanced Risk & Recall	Richer graph policies, confidence scoring, automated alerting and regulator workflows
Advanced Business Intelligence	Advanced distribution/consumer analytics, optimization and executive intelligence

4. Final Base Architecture

Recommended engineering stack: **Next.js/React consumer + stakeholder web UI** → **FastAPI backend** → **PostgreSQL + Redis + IPFS + Hyperledger Fabric Gateway** → **Fabric peers/orderer/chaincode**, with the Risk Propagator as a backend/domain service operating over the PostgreSQL lineage model.

The choice of FastAPI is an engineering recommendation rather than a requirement frozen by the supplied documents. It is recommended because the project needs rapid API development, graph/risk logic, evidence handling and easy future integration with analytical services. If the team already has stronger Spring Boot expertise, the backend can be implemented in Spring Boot without changing the architecture; the contracts and responsibilities below should remain unchanged.

1. Consumer / Stakeholder UI — registration, scan, operations, dashboard, feedback
2. Backend API — authentication, authorization, validation, orchestration and service coordination
3. PostgreSQL — authoritative operational/query model for application data and lineage
4. Redis — cache for hot traceability/history/risk views
5. IPFS — large evidence objects; CID + integrity reference linked to records
6. Fabric Gateway — submits/query trusted transactions against the consortium network
7. Fabric Chaincode — validates shared rules and state transitions
8. Fabric Ledger — tamper-evident record of accepted critical shared events
9. Risk Propagator — graph traversal and affected-scope computation outside chaincode

5. Technology Responsibility Matrix

Technology	Responsibility	Must NOT become
Next.js / React	Consumer and stakeholder UI	Business-rule authority
FastAPI	API, orchestration, validation, queries, integrations	Source of trusted cross-org truth
PostgreSQL	Operational data, lineage, incidents, feedback, read models	Blockchain replacement for trusted shared events
Redis	Fast cache for common reads and summaries	Source of truth
IPFS	Certificates, reports, images and large evidence	Authorization system or proof that evidence is truthful
Hyperledger Fabric	Permissioned shared trust/audit layer	General-purpose application database
Fabric Chaincode	Critical shared authorization/state/business rules	Graph analytics, ML, file storage or cache
Risk Propagator	Upstream/downstream impact analysis	Fabric transaction logic

6. Why Hyperledger Fabric

The project involves known supply-chain organizations rather than anonymous public participants. The research/blueprint recommends Hyperledger Fabric for the permissioned setting, and the PRD explicitly defines Fabric as the trusted data layer and chaincode as the trusted logic layer. ■filecite■turn10file5■L1-L1■

The Base Model should therefore use Fabric MSP identities, organization membership, peers and an ordering service. The pilot can keep the network simple — a small consortium with the minimum organizations required for the demo — while keeping the architecture extensible for later private data separation.

7. Identity Model

The most important engineering invariant is **one stable product identity + one reliable lineage model + controlled lifecycle events**. The QR is only a reference to an identity; it is not the identity itself. This distinction is explicitly present in the PRD. ■filecite■turn9file0■L3-L4■

Entity	Technical role	Key fields (minimum)
Organization	Known supply-chain participant	org_id, name, type, status, fabric_msp_id
User	Human/system actor	user_id, org_id, role_id, status
Role	Authorization context	role_id, permissions
Product	Product definition	product_id, name, category, product_type
Batch/Lot	Traceability group	batch_id, product_id, quantity, unit, current_state, owner_org
Unit	Individual package	unit_id, batch_id, serial/reference, current_state
Event	Meaningful business occurrence	event_id, type, actor, target, timestamp, fabric_tx_id
Lineage	Parent-child relationship	parent_id, child_id, relation_type, quantity
QR Credential	Reference / verification data	credential_id, unit_id, public_ref, status
Incident	Complaint/safety/quality signal	incident_id, unit/batch, category, severity, status
Evidence	Supporting file reference	evidence_id, CID, hash/commitment, type, owner
Accountability	Responsibility/performance signal	record_id, stakeholder, incident, level, score/signal
Risk/Recall Set	Computed affected scope	scope_id, incident_id, affected_nodes, status

8. Batch vs Unit Identity

Not every kilogram of raw material needs an individual QR. The system should primarily identify **batches/lots**. Unit-level identity becomes necessary when consumer-level verification, authenticity or precise recall is required — normally at final packaging. This preserves scalability while still allowing precise consumer incidents.

Example: W001 → F001 → B001 → B001-U0723. W001, F001 and B001 can be batch identities; B001-U0723 is the consumer unit. The same model supports a batch producing many units and a transformation combining multiple parent inputs.

9. Lifecycle State Machine

Freeze the initial state machine before writing chaincode. The PRD/TRD recommend a compact state set; not every product must use every state. ■filecite■turn10file1■L1-L1■

1. REGISTERED — identity created, basic information captured
2. VALIDATED — required validation passed
3. IN_TRANSIT — custody/transport operation started
4. RECEIVED — receiving stakeholder accepted the product
5. PROCESSED / TRANSFORMED — transformation created a child product/batch
6. AVAILABLE — product is eligible for downstream sale/use
7. UNDER_INVESTIGATION — issue is being investigated
8. BLOCKED — movement/sale is restricted
9. RECALLED — authorized recall action is active
10. CLOSED — incident/action completed

Every transition must include actor organization, actor role, current state, requested operation, target identity, timestamp, optional evidence and next state. Chaincode rejects invalid transitions. A QR scan alone never changes state; it is an interaction. A verified business operation changes state.

10. Fabric Network Design

For the SIH Base Model, use a minimal permissioned consortium. A practical pilot can represent Farmer/Producer, Manufacturer, Transporter, Retailer and Regulator/Authority as organizations or role groups depending on the team’s available infrastructure. The exact number of peer nodes can remain small for the demo; the logical organization boundaries must still be represented.

Fabric component	Base requirement
MSP / certificates	Every participating organization has an authenticated identity context
Peers	Maintain ledger state and endorse transactions according to network policy
Ordering service	Orders endorsed transactions for commitment
Channel	Use a simple shared channel for V1 unless a concrete confidentiality need requires separation
Chaincode	Focused lifecycle, authorization and critical event rules
Gateway	Backend uses Fabric Gateway/API rather than exposing Fabric directly to clients
Events	Backend listens for committed transaction events to update the operational model

The supplied research on Fabric-based traceability supports lifecycle management, transfers and quality/control rules, while the PRD explicitly keeps complex graph queries outside chaincode. ■filecite■turn10file15■L1-L1■

11. Chaincode Transaction Families

Transaction	Purpose	Core validation
registerBatch	Create a new source/accepted batch	Authorized org, unique identity, valid input
validateBatch	Mark validation result	Authorized validator, correct state
receiveBatch	Accept custody/receipt	Authorized receiver, transferable state
transferBatch	Record custody transfer	Sender/receiver authorization, valid state
createTransformation	Create child batch and lineage	Authorized processor, valid parents, quantities
createUnit	Create consumer unit identity	Authorized packaging/manufacturer, valid batch
blockBatch / blockUnit	Restrict further movement	Authorized role + reason
startInvestigation	Mark safety/quality investigation	Authorized incident workflow
createRecallAction	Record authorized recall/block action	Authorized authority + affected scope
closeIncident	Close after resolution	Authorized reviewer + resolution evidence

Chaincode must validate and commit; it should not call Redis, upload to IPFS, run graph traversal or perform complex analytics. The research/blueprint and TRD explicitly separate these responsibilities. ■filecite■turn10file1■L1-L1■

12. Fabric ↔ PostgreSQL Consistency Model

Do not create uncontrolled dual writes. The recommended pattern is **Fabric transaction accepted first** → **Fabric commit event observed** → **PostgreSQL operational/read model updated** → **Redis cache invalidated/refreshed**. This makes Fabric the trust anchor for critical shared events while PostgreSQL remains the query-optimized application model. ■filecite■turn10file4■L1-L1■

1. Client requests a protected business operation
2. Backend authenticates + authorizes + validates request
3. Backend submits transaction through Fabric Gateway
4. Chaincode validates role, state and business rule
5. Fabric orders/commits transaction
6. Backend receives committed event / transaction result
7. PostgreSQL stores the operational/read-model representation with fabric_tx_id
8. Redis invalidates or refreshes affected cache keys
9. Failure in synchronization is reconciled from Fabric events; no manual fake history is created

13. PostgreSQL Technical Model

PostgreSQL is the application’s operational and query layer. It should contain enough information to reconstruct product history, lineage, incidents and affected scope quickly. The blockchain ledger remains the trusted record for critical shared events, but PostgreSQL is where the Risk Propagator and dashboards perform practical queries.

Table	Important columns / relationships
organizations	id, name, type, fabric_msp_id, status
users	id, organization_id, role_id, auth_subject, status
roles / permissions	role_id, permission_code
products	id, name, product_type, category
batches	id, product_id, parent/reference metadata, quantity, state, owner_org_id
units	id, batch_id, serial/reference, state, qr_credential_id
lineage_edges	parent_batch_id, child_batch_id, relation_type, quantity, created_event_id
custody_events	id, batch/unit, from_org, to_org, event_type, timestamp, fabric_tx_id
events	id, type, actor_org, actor_user, target, state_before, state_after, fabric_tx_id
qr_credentials	id, unit_id, public_reference, credential_status, binding_metadata
incidents	id, unit_id/batch_id, category, severity, status, source, created_at
feedback	id, incident_id, category, description, evidence_ref, location_granularity, verification_status
accountability_records	id, incident_id, stakeholder_id, level, signal_value, reason
evidence	id, CID, content_hash, type, access_class, linked_entity
risk_scopes	id, incident_id, scope_status, generated_at
risk_scope_nodes	scope_id, entity_type, entity_id, impact_status
recall_actions	id, incident_id, scope_id, action_type, status, authorized_by
audit_log	actor, action, target, timestamp, result, request_id

14. Lineage Data Structure

Do not use the blockchain ledger as a graph database. Store lineage in PostgreSQL as indexed adjacency relationships. At minimum, index parent_batch_id, child_batch_id, product_id and batch state. This supports efficient BFS/DFS-style upstream and downstream traversal.

For many-to-one transformations, store multiple parent edges pointing to one child. For one-to-many, store one parent connected to multiple children. Where quantities matter, store the quantity contributed by each parent so recall analysis can distinguish direct and derived relationships.

15. Redis Strategy

Redis is a performance layer only. Recommended cache keys include **qr:{public_ref}**, **trace:{unit_id}**, **batch:{batch_id}:summary**, **risk:{incident_id}** and dashboard summary keys. Every cached object must have a TTL/invalidation policy and must be rebuildable from PostgreSQL/Fabric.

Redis should not contain unique information that cannot be reconstructed. If Redis is deleted, the application must remain correct after cache warm-up.

16. IPFS Evidence Strategy

Certificates, laboratory reports, inspection reports, images and other large evidence remain off-chain. The backend uploads the object to IPFS, receives a CID, stores the CID plus a cryptographic integrity reference in PostgreSQL, and associates that reference with the relevant event/incident/product/batch. Important evidence must use pinning or a persistence/cluster strategy; IPFS alone does not guarantee permanent availability. ■filecite■turn10file3■L1-L1■

17. Outer QR Architecture

The outer QR should contain only a stable public reference or URL token. It should never contain the complete product history. Example: **https://trace.example/p/{public_ref}**. The backend resolves the reference to the registered product/unit and returns the permitted consumer-safe traceability view.

1. Consumer scans visible QR
2. UI sends public reference to backend
3. Backend resolves reference → unit/batch
4. Backend checks current public status and permitted history
5. Redis may serve a cached traceability view
6. PostgreSQL supplies operational history/lineage
7. Consumer sees source/processing/transfer/status information

A copied outer QR can therefore still resolve to a genuine history. That is acceptable because the outer QR is explicitly a traceability reference, not the physical authenticity proof.

18. Inner Authenticity Layer — Base Interface

The project requires a second concealed/tamper-evident credential concept. The exact production-grade cryptographic construction should not be frozen prematurely. The Base Model should nevertheless freeze the **interface and data relationship** so the later security module can be plugged in without redesigning identity.

Base element	Requirement
unit_id	Stable unit identity already created by the core model
outer_public_ref	Visible QR reference linked to the unit
inner_credential_id	Reference to the concealed credential record
binding_status	ACTIVE / SUSPENDED / REVOKED / UNVERIFIED
verification API	verifyCredential(unit_id, presented_credential)
verification result	Traceability Verified and Physical Authenticity Check shown separately

Do not claim that SHA-256 vs SHA-512, or any pair of different hashes, automatically prevents copying. The security property must come from cryptographic binding, credential issuance, replay/copy resistance and suitable physical/tamper-evident packaging. The supplied project documents explicitly make this caveat.
■filecite■turn10file10■L1-L1■

For the SIH Base Model, a controlled prototype can implement a deterministic unit-bound credential verification interface, while the final physical/cryptographic design is hardened later.

19. Consumer Feedback & Accountability — Base V1

Feedback is not a star-rating system. It is a **risk signal + accountability workflow**. The PRD defines complaint categories, unit/batch linkage, transaction context, status lifecycle and escalation rules. ■filecite■turn9file0■L4-L5■

Feedback field	Requirement
incident_id	Unique incident reference
unit_id / batch_id	The product context; unit preferred when available
category	Damage, quality, contamination, labeling, expiry, authenticity, other
severity	Low / Medium / High / Critical policy
description	Consumer-provided explanation
evidence	Optional image/document via IPFS
timestamp	Server-side authoritative time
transaction/retailer context	Associated when available
location	Consent-based and appropriate granularity
verification_status	UNVERIFIED / REVIEWING / VERIFIED / REJECTED
status	SUBMITTED / UNDER_REVIEW / ESCALATED / RESOLVED / CLOSED

20. Accountability Algorithm — Rule-Based V1

The nearest relevant layer receives the first accountability signal, but the system must not declare that layer guilty. Repeated complaints are aggregated using similarity, severity, distinct units, retailers/transactions, geography, time window and verification status. This matches the PRD's signal-and-escalation model. ■filecite■turn10file4■L1-L1■

1. Level 1 — Local Complaint: assign initial review to nearest relevant layer
2. Level 2 — Repeated Pattern: same/similar issue appears for the same product/batch
3. Level 3 — Distributed Batch Pattern: similar issue spans distinct units/retailers/locations
4. Level 4 — Verified Systemic Issue: evidence/review confirms batch/process problem
5. Escalate only when configured conditions are satisfied; update accountability records and notify authorized stakeholders

A V1 rule can compute a pattern score from weighted factors: complaint similarity, severity, distinct units, distinct retailers, geographic spread, time concentration and verification state. Exact numeric weights should be configurable in the database rather than hard-coded in chaincode.

21. Feedback Anti-Abuse Controls

A unit-linked QR strengthens association with a real product, but it does not prove that the complaint is truthful. Base controls should include rate limiting, duplicate feedback policy per unit, evidence support, server-side timestamps, verification status and suspicious repeated-submission flags. The supplied TRD explicitly identifies this as a key control. ■filecite■turn10file3■L1-L1■

22. Risk Propagator — Base Version

The Risk Propagator is a backend service that turns lineage into safety action. It receives an incident or confirmed defect and constructs an affected-scope set. Upstream traversal follows parent relationships; downstream traversal follows child relationships and distribution/custody relationships. ■filecite■turn10file4■L1-L1■

Input	Meaning
incident_id	The triggering complaint/incident
target_unit/batch	Where the issue was observed
severity	Priority of investigation
verification_state	Unverified / reviewed / confirmed
traversal_policy	How far and which relationship types to follow
Output	**Meaning**
source_candidates	Potential upstream parent materials
affected_descendants	Potentially affected downstream batches/units
locations	Retailers/destinations associated with affected nodes
stakeholders	Organizations that need investigation/notification
scope_status	POTENTIAL / REVIEWED / CONFIRMED

Important: graph connectivity means potentially affected, not automatically contaminated or guilty. Final recall/confirmed impact requires authorized review. This prevents the Risk Propagator from turning a graph relationship into an unsupported factual claim.

23. Base Risk Traversal Logic

1. Receive incident + target unit/batch
2. Map target to its batch/product
3. Traverse parent edges upstream until source/root or configured boundary
4. Traverse child edges downstream to derived batches and units
5. Join custody/distribution relationships to find retailers and locations
6. Remove duplicates and classify nodes as potential impact
7. Create risk_scope and risk_scope_nodes records
8. Return source candidates + affected scope + stakeholders
9. Allow authorized user to review and convert potential scope into action

Implementation recommendation: use PostgreSQL recursive CTEs for straightforward lineage traversal initially. If graph complexity becomes large later, the risk engine can evolve independently without changing the core data model.

24. Basic Block / Recall Workflow — Base

1. Incident created
2. Severity and evidence state classified
3. Initial accountability assigned
4. Related complaints aggregated
5. Risk Propagator computes potential affected scope
6. Authorized stakeholder reviews scope
7. Batch/unit is BLOCKED or marked UNDER_INVESTIGATION
8. Authorized recall action is created when required
9. Stakeholders/locations are notified through the application
10. Action and final resolution are recorded as auditable events

The Base Model supports targeted recall scope generation. It does not replace official regulatory recall procedures or automatically make a final legal/regulatory decision. The PRD explicitly places final action within authorized workflows. ■filecite■turn10file4■L1-L1■

25. Basic Stakeholder Dashboard — Base

Business Intelligence is deferred, but a basic operational dashboard is part of the Base Model because the team needs a usable interface to operate and demonstrate the system. The advanced BI module will later expand this into deeper analytics and optimization.

Base dashboard	Minimum content
Supply Chain Overview	Source → processing → manufacturing → transport → retail → consumer status
Batch Detail	Identity, current state, parent/child lineage, events, destinations
Incident View	Severity, evidence, status, accountability layer, escalation
Risk View	Upstream sources, downstream affected scope, locations, review state
Recall View	Blocked/recalled units/batches, scope, action status
Audit View	Who performed what, when, against which entity

26. Access Control

Base security includes RBAC and organization-aware authorization even though the **Advanced Privacy** module is deferred. Consumers see only consumer-safe traceability. Stakeholders see authorized operational information. Sensitive commercial evidence is not globally exposed by default.

A future privacy module may add private data collections, selective disclosure and stronger cross-organization isolation, but Base Model APIs must already enforce an authorization boundary so that later privacy hardening does not require rewriting the application.

27. Backend API Contract — Base

The exact URL naming can evolve, but the service boundaries should be frozen now. Clients must never directly modify Fabric or trusted history.

API group	Representative operations
Auth / Organizations	login, getMe, createOrganization, assignRole
Products	createProduct, getProduct
Batches	createBatch, validateBatch, getBatch
Units	createUnit, getUnit
Events	getEvents, submitBusinessOperation
Lineage	getParents, getChildren, getLineage
QR	resolveQR, generateQR, verifyCredential
Evidence	uploadEvidence, getEvidenceMetadata
Feedback	submitFeedback, getIncident, updateIncidentStatus
Accountability	getAccountability, evaluateEscalation
Risk	propagateRisk, getRiskScope
Recall	blockBatch, createRecallScope, createRecallAction
Dashboard	getSupplyChainOverview, getBatchDashboard, getIncidentDashboard
Audit	getAuditTrail

28. Representative End-to-End Transaction — Receive Batch

1. Manufacturer scans/enters batch reference
2. Backend resolves batch and checks PostgreSQL current read model
3. Backend authenticates manufacturer user and verifies role
4. Backend submits receiveBatch to Fabric Gateway
5. Chaincode checks organization, current state and transition rule
6. Fabric commits the transaction
7. Backend receives committed result/event
8. PostgreSQL records receive event + new state + fabric_tx_id
9. Redis invalidates the batch/traceability cache
10. UI returns successful receipt confirmation

29. Representative Consumer Incident Transaction

1. Consumer scans inner/consumer credential or outer QR
2. Backend resolves unit/batch
3. Consumer submits issue category + description + optional evidence
4. Evidence is uploaded to IPFS if present
5. Feedback/incident is stored in PostgreSQL with evidence CID
6. System assigns initial accountability context
7. Pattern evaluator checks related complaints
8. If threshold is reached, escalation record is created
9. Risk Propagator computes upstream/downstream potential impact
10. Authorized stakeholder reviews and can block/recall
11. Critical action is committed through Fabric and reflected in PostgreSQL

30. Security Requirements — Base

Control	Base requirement
Authentication	All stakeholder operations require authenticated identity
RBAC	Permissions mapped to organization + role
Chaincode authorization	Critical state-changing operations validated again at trust layer
Input validation	Validate product IDs, quantities, states, roles and evidence metadata before commit
Audit	Critical admin, supply-chain, incident and recall actions logged
Evidence integrity	Store CID + content hash/commitment and retain metadata
Feedback abuse	Rate limit, duplicate policy, verification state and evidence support
QR security	Outer QR is not authenticity proof
Inner credential	Bound to unit; final crypto design separately tested
Secrets	Never place Fabric private keys, DB passwords or API secrets in frontend code
Data exposure	Consumer-safe views separated from internal operational views

31. Non-Functional Requirements

Integrity: critical business events must be tamper-evident and auditable. **Availability:** basic QR traceability should continue to work even if Risk Propagation is temporarily unavailable. **Performance:** cache frequent lookups. **Scalability:** keep graph traversal outside chaincode. **Observability:** log requests, transaction IDs, failures, scans, state changes and incidents. **Modularity:** future modules must consume the same identity/event/lineage foundation.

32. Testing Strategy

Test layer	Minimum tests
Unit tests	State transitions, lineage functions, escalation rules, QR resolution, risk traversal
Chaincode tests	Unauthorized operations, invalid states, duplicate IDs, blocked transfers, valid transformations
API tests	Auth/RBAC, CRUD, transaction orchestration, failure/retry behavior
Integration tests	Backend ↔ Fabric, Fabric event → PostgreSQL, PostgreSQL ↔ Redis, IPFS upload/reference
End-to-end	Farmer → producer → manufacturer → transporter → retailer → consumer → incident → recall scope
Security tests	RBAC bypass, replay/duplicate feedback, QR tampering, credential misuse, secret exposure
Failure tests	Fabric unavailable, PostgreSQL unavailable, Redis cleared, IPFS upload failure, event sync interruption

33. Recommended Implementation Order

The order below is intentionally dependency-driven. Do not build UI features that depend on identities/lineage before those foundations are stable.

1. Freeze organizations, roles, permissions, entities and lifecycle states
2. Create PostgreSQL schema + migrations + seed data
3. Set up Fabric network, MSP identities, peers/orderer and Gateway
4. Implement chaincode skeleton + core state transition rules
5. Implement product/batch/unit creation
6. Implement lineage creation and queries
7. Implement receive/transfer/transform business flows
8. Implement Fabric event listener → PostgreSQL read-model synchronization
9. Implement outer QR generation/resolution and consumer traceability page
10. Implement IPFS evidence upload + CID/integrity linkage
11. Implement minimum inner credential interface/prototype
12. Implement consumer feedback/incident reporting
13. Implement accountability rules + configurable escalation
14. Implement first Risk Propagator
15. Implement basic block/recall workflow
16. Implement basic stakeholder dashboard + audit view
17. Add Redis cache and performance tests
18. Run full end-to-end scenario and harden failures/security

34. Base Model Definition of Done

Acceptance condition	Pass criteria
Traceability	One sample product is traceable source → final unit
Lineage	Parent/child transformations reconstruct correctly
Lifecycle	Invalid transitions rejected; valid transitions committed
Fabric	Critical shared events have committed transaction IDs
PostgreSQL	Operational/read model is queryable and reconciles with Fabric
Redis	Common QR/history/risk views cache correctly and can be rebuilt
IPFS	Evidence is retrievable via stored CID/reference
Outer QR	Consumer can see permitted product journey
Inner authenticity	Base interface/prototype can distinguish traceability from authenticity result
Feedback	Complaint is linked to correct unit/batch and stored with status
Accountability	Initial layer assigned; repeated patterns can escalate using configurable rules
Risk Propagator	Upstream + downstream affected scope generated
Recall	Authorized user can create a targeted block/recall scope
Dashboard	Stakeholder can inspect batch, incident, risk and audit information
Audit	Critical actions are attributable to actor + timestamp + transaction/request IDs

35. Future Module Boundary

After Base Model V1 is stable, the following modules can be attached without changing the core identity/lineage model: **M1 Advanced Risk & Recall**, **M2 Advanced Business Intelligence**, **M3 EPCIS**, **M4 Event Streaming/Kafka**, **M5 IoT**, **M6 Advanced Privacy**, and **M7 AI/ML**. The inner-authenticity mechanism can also be hardened as a dedicated security module once the Base identity and credential interfaces are stable.

36. Final Engineering Decision

The team should freeze the Base Model around three invariants: ***stable identity, reliable lineage, controlled lifecycle events.*** *Everything else — feedback, accountability, Risk Propagation, recall, dashboards, evidence and later advanced modules — depends on those foundations.*

The final Base Model is therefore: Organizations/RBAC → Product/Batch/Unit Identity → Lifecycle → Lineage → Fabric + Chaincode → PostgreSQL → Redis → IPFS → Outer QR + Base Inner-Credential Interface → Feedback → Accountability → Core Risk Propagator → Basic Block/Recall → Basic Dashboard/Audit.

The deferred modules are deliberately excluded from Base Model development: **EPCIS, Event Streaming, IoT, Advanced Privacy, AI/ML, Advanced Risk & Recall, and Advanced Business Intelligence.**

The supplied research and project blueprint support the core technology direction — Fabric/chaincode, IPFS, permissioned traceability and hybrid storage — while the exact inner-authenticity construction and bidirectional Risk Propagator remain project-specific engineering features that require implementation/testing rather than being treated as already-proven mechanisms. ■filecite■turn10file9■L1-L1■

Engineering slogan: *Collect → Validate → Record trusted event → Preserve evidence → Connect lineage → Propagate risk → Verify → Act.*